

CASO PRÁCTICO 1 – UNIDAD 4

Mantenimiento del estado y almacenamiento de información en aplicaciones web

1. Introducción

En este caso práctico se ha desarrollado una aplicación web tipo tienda online cuyo objetivo principal es aplicar los conceptos vistos en la Unidad 4 relacionados con el mantenimiento del estado y el almacenamiento de información.

La aplicación permite al usuario:

- Mantener un carrito de la compra mientras navega entre distintas páginas.
- Conservar la preferencia de idioma incluso tras recargar o cerrar el navegador.

Para ello, se han utilizado **sesiones PHP** en el servidor y **LocalStorage** en el lado cliente mediante JavaScript, garantizando una experiencia coherente y persistente.

2. Mantenimiento del estado en el servidor mediante sesiones PHP

Para gestionar el estado del carrito de la compra se han utilizado **sesiones PHP**, lo que permite almacenar información asociada al usuario entre distintas peticiones HTTP.

2.1 Inicialización de la sesión y del carrito

Al inicio de la aplicación se comprueba si existe un carrito en la sesión del usuario. En caso contrario, se inicializa como un array vacío. Esto asegura que el carrito esté disponible durante toda la navegación.

```
session_start();  
// Iniciar carrito si no existe  
if (!isset($_SESSION['cart'])) {  
    $_SESSION['cart'] = [];  
}
```

2.2 Añadir productos al carrito

Cuando el usuario añade un producto, se recibe el identificador mediante un formulario POST.

Si el producto ya existe en el carrito, se incrementa su cantidad; si no, se añade por primera vez.

```
if (isset($_POST['add_to_cart'])) {  
    $productId = intval($_POST['product_id'] ?? 0);  
    if ($productId > 0) {  
        if (!isset($_SESSION['cart'][$productId])) {  
            $_SESSION['cart'][$productId] = 1;  
        } else {  
            $_SESSION['cart'][$productId]++;  
        }  
    }  
}
```

Gracias al uso de sesiones PHP:

- El carrito se mantiene entre páginas.
- La información no se pierde al recargar.
- El estado del usuario se gestiona de forma centralizada en el servidor.

3. Almacenamiento de preferencias del usuario con Web Storage

Para almacenar la preferencia de idioma de la interfaz se ha utilizado **LocalStorage**, que forma parte de la API Web Storage.

Este tipo de almacenamiento es adecuado para preferencias personales del usuario que no necesitan guardarse en el servidor.

3.1 Guardado del idioma en LocalStorage

Al pulsar el botón de cambio de idioma, se alterna entre español e inglés y se guarda la preferencia en el navegador.

Tras el cambio, la página se recarga para aplicar el idioma seleccionado de forma global.

```
const langBtn = document.getElementById("lang_toggle");
if (langBtn) {
  let idioma = localStorage.getItem("idioma") || "es";
  langBtn.textContent = idioma.toUpperCase();

  langBtn.addEventListener("click", () => {
    idioma = idioma === "es" ? "en" : "es";
    localStorage.setItem("idioma", idioma);
    langBtn.textContent = idioma.toUpperCase();
    location.reload();
  });
}
```

4. Recuperación automática de la información almacenada

Al cargar cada página, la aplicación recupera el idioma almacenado en LocalStorage y actualiza automáticamente los textos de la interfaz.

4.1 Aplicación de traducciones según el idioma

Se utiliza un objeto de traducciones y una función que recorre sus claves, actualizando los elementos HTML por su identificador.

```
function applyTranslations() {
  const lang = localStorage.getItem('idioma') || 'es';
  const dict = translations[lang];

  for (const key in dict) {
    const el = document.getElementById(key);
    if (el) el.innerHTML = dict[key];
  }
}
```

4.2 Ejecución al cargar la página

Esta función se ejecuta automáticamente cuando la página termina de cargarse, garantizando que el idioma se aplique de forma consistente.

```
document.addEventListener('DOMContentLoaded', applyTranslations);
```

5. Uso del estado almacenado en la interfaz

El estado del carrito se refleja visualmente en la interfaz, mostrando en todo momento el número total de productos almacenados en la sesión del usuario.

```
<span class="cart-count">  
  <?php echo array_sum($_SESSION['cart'] ?? []); ?>  
</span>
```

Esto permite al usuario comprobar el contenido de su carrito sin necesidad de acceder a una página específica.

6. Conclusión

En este caso práctico se han aplicado correctamente los mecanismos de mantenimiento del estado y almacenamiento de información estudiados en la unidad.

El uso de **sesiones PHP** permite conservar el estado del carrito de la compra durante la navegación, mientras que **LocalStorage** se utiliza para almacenar y recuperar las preferencias del usuario.

La combinación de ambas técnicas proporciona una experiencia de usuario coherente, persistente y funcional, cumpliendo los objetivos planteados para este ejercicio.

7. Recursos

- Mimo App y Enki App: práctica de HTML, CSS y JavaScript.
- CodePen: comprobación del funcionamiento del código.
- Program with Gio – PHP Tutorial (YouTube).